

SMART INDIA HACKATHON 2026 · PROBLEM STATEMENT 26187 · TEAM SW-73
(BYTEFORGE)

DRISHTI—BOP

AI-based intelligent video analytics for border surveillance, built entirely on existing CCTV infrastructure — system architecture reference.

Stack Python 3.11 · FastAPI · React/TS · PostgreSQL · MediaMTX

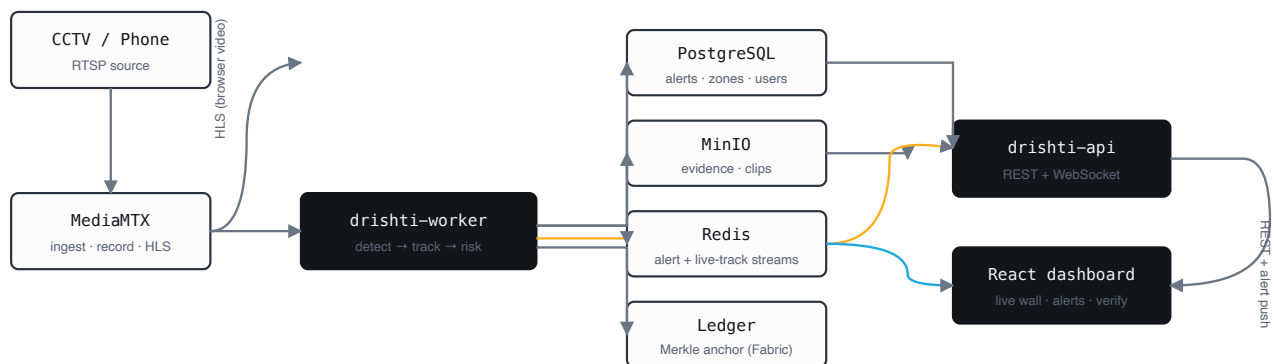
Cost ₹0 marginal — every model and service is free and self-hostable

01 What this system does

Border Out Posts already have CCTV. Those cameras do two things: show live video, and record it. Everything else — noticing an intrusion, recognising a vehicle, keeping an audit trail — is a human in a room, and the failure mode of a tired human is silent.

DRISHTI-BOP bolts onto the RTSP streams that already exist. It detects and tracks what a camera sees, reasons about it against operator-defined zones, scores every candidate event with an additive model whose terms are all visible, raises debounced evidence-backed alerts, and writes a tamper-evident audit trail anchored to a permissioned ledger. It never acts autonomously — every alert is a recommendation a human adjudicates.

02 End-to-end data flow



Amber path: alert stream (Redis → API → dashboard, durable). Blue path: live-track overlay (Redis → API → dashboard, best-effort, cosmetic-but-real).

03 The worker pipeline, stage by stage

STAGE	WHAT HAPPENS	MODULE
Ingest	Pulls RTSP with a watchdog + reconnect loop; recording continues even if analytics crash	ingest.py
EVQM	Measures brightness, contrast, blur, fog, noise; picks a processing profile with hysteresis so a passing headlight can't flip it every frame	evqm.py
Enhance	Runs on the model-scale image only, after letterboxing — never touches the frame stored as evidence	enhance.py
Detect	YOLO11n via ONNX Runtime (CPU / CUDA / TensorRT / CoreML, auto-selected); warms up before the worker reports ready	detect/onnx_yolo.py
Track	ByteTrack, reimplemented in-repo — a two-pass (high- then low-confidence) association that survives brief occlusion	track.py
Geometry + Rules	Tests each track's foot-point against operator-defined zones (tripwire / area / mask); fires only on zone <i>entry</i> , not every frame inside	rules.py, geometry.py
Risk	Additive scoring — every signal's contribution is visible and the terms sum exactly to the score, on purpose	risk.py
Debounce	Per-track/zone cooldown so a lingering subject doesn't spam repeat alerts	rules.py (Debouncer)
Evidence	RFC 8785 canonicalisation → SHA-256 hash, computed at assembly time before storage	evidence.py
Ledger	Hashes batched into a Merkle tree and anchored; ledger unavailability never blocks the alert itself	merkle.py, ledger/
Sinks	Postgres (durable) + Redis (live push) + MinIO (media); a failing sink spools to disk rather than losing the alert	sinks.py

04 Explainable risk scoring

Every alert's score is a sum of independent signals — nothing is a black box. A live example, from an actual alert this system raised:

SIGNAL	WEIGHT	WHY
ZONE_INTRUSION	+40.0	A confirmed person's foot-point entered a restricted zone for the first time
SHORT_TRACK	-12.0	The object had only been tracked for a few frames — confidence is lowered rather than suppressed outright
Total	28.0 → low	Severity bands: ≥80 critical · ≥60 high · ≥40 medium · ≥20 low

A track only reaches the rule engine at all once it has `hits ≥ 3` and `age ≥ 8 frames` — a single noisy detection never becomes an alert.

05 Tamper-evident evidence chain

1 • CANONICALISE

Every evidence document is serialised with RFC 8785 (JCS) — deterministic key ordering and number formatting, so semantically identical JSON always produces byte-identical output.

2 • HASH

SHA-256 over the canonical bytes, computed once at assembly time in the worker — before the record ever reaches storage.

3 • BATCH

Hashes are grouped into a Merkle tree with domain-separated leaf/node prefixes, giving every alert a compact inclusion proof.

4 • ANCHOR

The Merkle root is written to a permissioned ledger (Hyperledger Fabric in production; a real hash-chained append-only log by default) — anchoring never blocks the alert firing.

Anyone — not just this dashboard — can verify a record independently: re-canonicalise, re-hash, compare. A single altered field is caught and reported as a field-level diff, never a silent mismatch.

06 Live camera overlay (real-time HUD)

Every processed frame, the worker publishes each currently-confirmed track's real box, class, and a measured speed (from the track's own foot-point motion) to a lightweight Redis stream. The API relays it over `/ws/live/{camera_id}`, filtered server-side per camera, and the dashboard draws it directly onto the live video — genuine model output, not a simulation. This feed is deliberately best-effort: unlike an alert, losing one frame of it is invisible a moment later.

07 The free stack

LAYER	CHOICE	LICENCE
Detection	YOLO11n/s (ONNX) · RT-DETR-R18 fallback	AGPL-3.0 · Apache-2.0
Tracking	ByteTrack, reimplemented in-repo	MIT (algorithm)
ANPR	YOLO11n plate detect + PaddleOCR v4	AGPL-3.0 · Apache-2.0
Faces (opt-in, off by default)	InsightFace SCRFD + ArcFace	MIT
Inference runtime	ONNX Runtime (CPU / CUDA / TensorRT / CoreML)	MIT
Database	PostgreSQL 16 + pgvector	PostgreSQL Licence
Object store	MinIO	AGPL-3.0
Media server	MediaMTX	MIT
Ledger	Hyperledger Fabric 2.5	Apache-2.0
API	FastAPI	MIT
Dashboard	React + Vite + Tailwind	MIT

Every model runs locally from a vendored weight file — no network call at inference time, ever. ₹0 marginal cost end to end.

08 Design principles that shaped every decision

- Evidence is always the original, unenhanced frame — enhancement is for the model, evidence is for the record
- Every alert's risk contributions sum exactly to its score, or it is not raised
- False alarms are the enemy — thresholds and debounce err toward suppression
- Face analytics defaults to disabled; non-matching embeddings are destroyed at track close
- Plate text is stored as HMAC — a database leak must not be identifying
- Recording survives an analytics crash, always
- No automated response of any kind — the system recommends, a human adjudicates
- Detections are always mapped back to original frame coordinates
- Ledger unavailability never blocks alerting — the anchor is queued instead
- Every model runs from a vendored weight file — no paid API, ever

DRISHTI-BOP · Smart India Hackathon 2026 · Problem Statement 26187 · Team SW-73 (ByteForge). Full specification: [docs/BUILD_SPEC.md](#).
Repository: github.com/Abuhanzala1234/F4sight.